

Playwright POC

Capgemini | beazley

Objectives

- Automate & run few functional test cases of GW Policy Center using Playwright.
- Automate & run Customized API Testing using Playwright.
- Automate & run the Cloud API Testing through Integration Gateway using Playwright.
- Test Data Parameterization – From External Source (Excel)
- Pros & Cons
- Appendix

Note: **Guidewire PC version :** JASPER
LoB: D&O

GW PC – Functionalities Validated with Playwright tool

Functionalities Tested:

Login Functionality

- Successful Login
- Failed Login

Account Creation for Person

- Account Creation with full data
- Mandatory Field Filled Check
- Name Of The Account Holder Already Exist
- Verify Exclusive Entry: Person or Company Details

Account Creation for Company

- Account Creation with UK data
- Account Creation with US data

Policy Creation for D&O insurance LOB

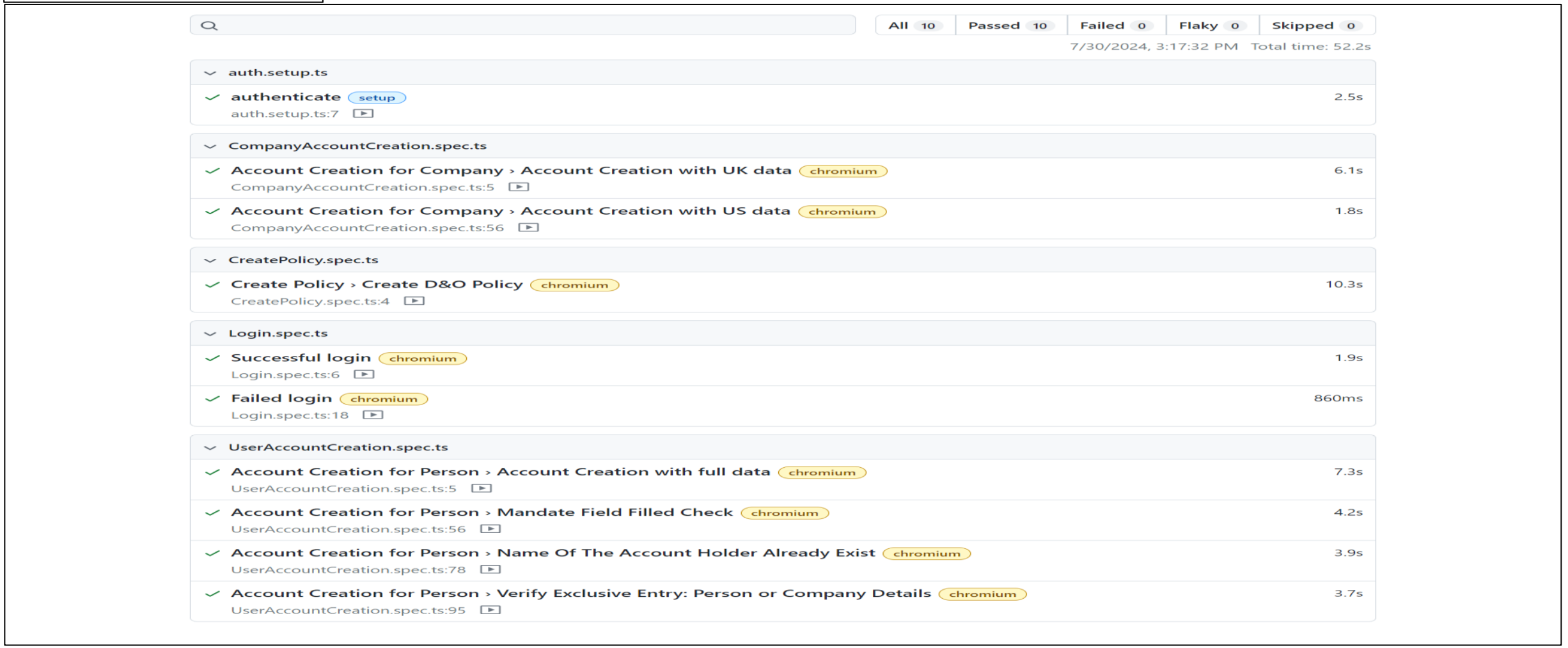
- Create D&O Policy(Submission, Quote, Bind and Issue)

Test Data:

	A	B	C
1	Fields	Values	Assertion Values
2		**PersonAccount**	
3	firstName	William	
4	lastName	Oliver	
5	country	United Kingdom	
6	town	London	
7	postalCode	E1W 2RG	
8	accountType	Person	
9	homePhone	+44 7975 111111	
10	workPhone	+44 7975 222222	
11	mobilePhone	+44 7975 333333	
12	primaryPhone	Home	
13	primaryEmail	williumOliver@gmail.com	
14	errorCodeMissingFilel		Address 1 : Missing required field
15	addressLine1	London	
16	addressType	Home	
17	organization	ACV Property Insurance	
18	producerCode	ProducerCode:16	
19	accountHolderName		William Oliver
20			
21		**companyAccount**	
22	CompanyName	Quantum Leap	
23	account	Company	
24	companyOfficePhone	+44 7975 333333	
25	companyPrimaryEmail	quantumLeap@outlook.com	
26	companyServiceTier	Gold Customer	
27	companyCountry	United Kingdom	
28	companyAddress1	London	
29	companyTown	London	
30	companyPostalCode	E1W 2RG	
31	companyAddressType	Business	
32	companyOrgType	Common ownership	
33	companyErrorMsg		Please specify either a company name or a person name, but not both.

Test Execution Report

Execution Report:



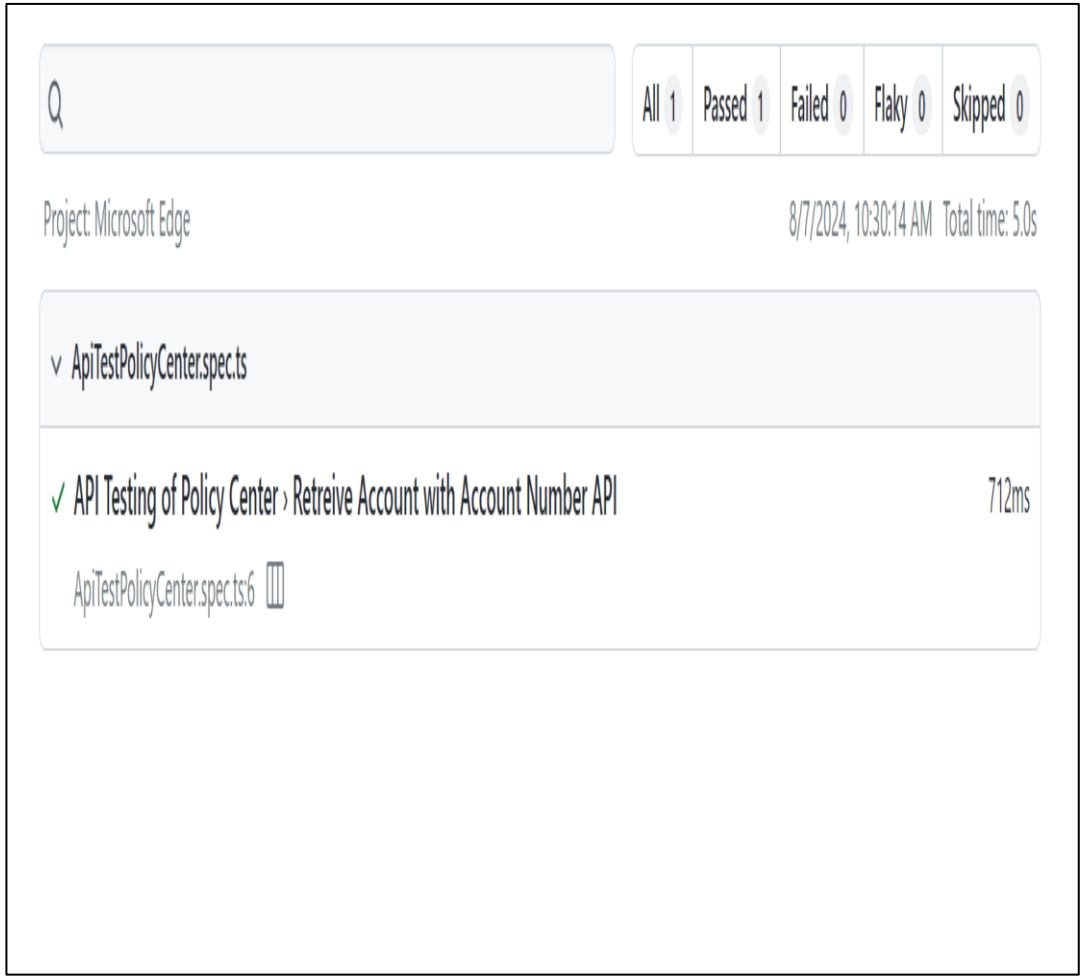
Customized API Testing with Playwright- 1

Customized API functions executed with Playwright:

- Retrieve the account API

(A custom cloud API has been developed in GWPC. Playwright script was developed and executed to Retrieve to view account summary)

Execution Report:



Customized API Testing with Playwright - 2

Purpose of this is to call customized API (Retrieve account API) and get response.

Retrieve account API Request and Response JSONs:

Request :
Scenario : Retrieve Account from AcctID - Policy Center
Method : GET
Endpoint URL :
http://localhost:8180/pc/rest/beazley/account/C000456354

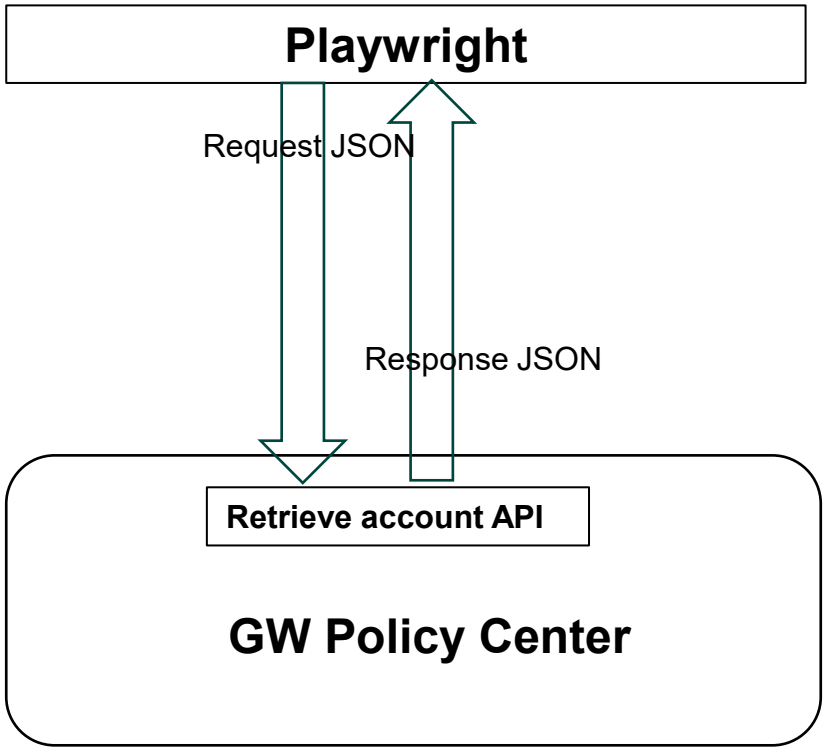
Response JSON:

getAccountfromAcctNumber.json

C: > Users > sillinda > AppData > Local > Microsoft > Windows > INetCache > Content.Outlook > UW3TGMEE > {} getAccountfromAcctNumber.json > ...


```
{
  "count": 1,
  "data": [
    {
      "attributes": {
        "accountHolder": {
          "displayName": "Wright Construction",
          "id": "test_pc:1",
          "type": "AccountContact",
          "uri": "/account/v1/accounts/pc:ds:1/contacts/test_pc:1"
        },
        "accountNumber": "C000212105",
        "accountStatus": {
          "code": "Active",
          "name": "Active"
        },
        "businessOperationsDescription": "business description",
        "createdDate": "2024-06-04T04:22:43.346Z",
        "frozen": false,
        "id": "pc:ds:1",
        "industryCode": {
          "code": "1522",
          "description": "General Contractors-Residential Building, Other Than Single-Family (except hotel and motel\n      construction)",
          "id": "SIC:1522"
        },
        "numberOfContacts": "8",
        "organizationType": {
          "code": "commonownership",
```


Flow Diagram:



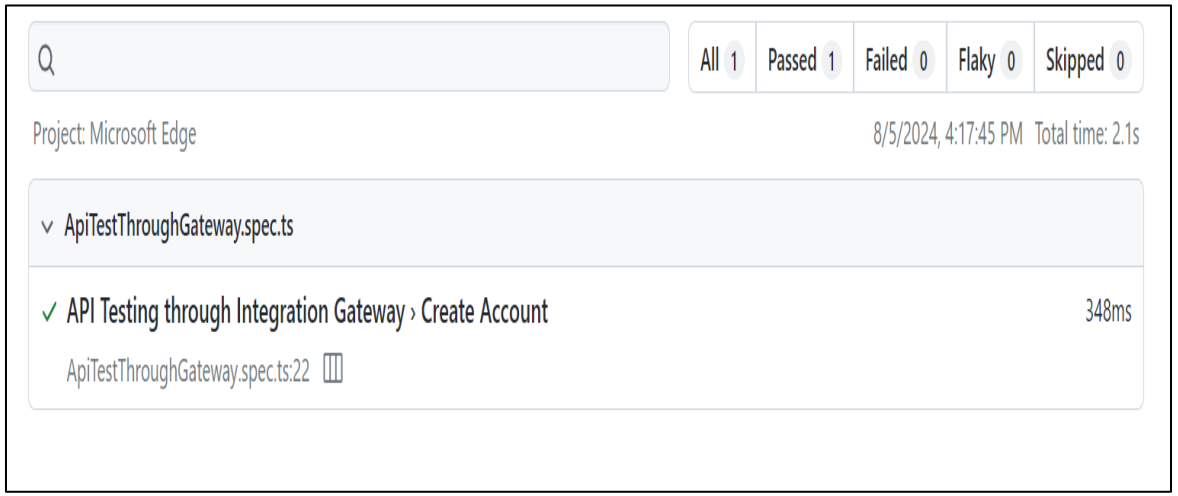
Cloud API Testing through Integration Gateway with Playwright - 1

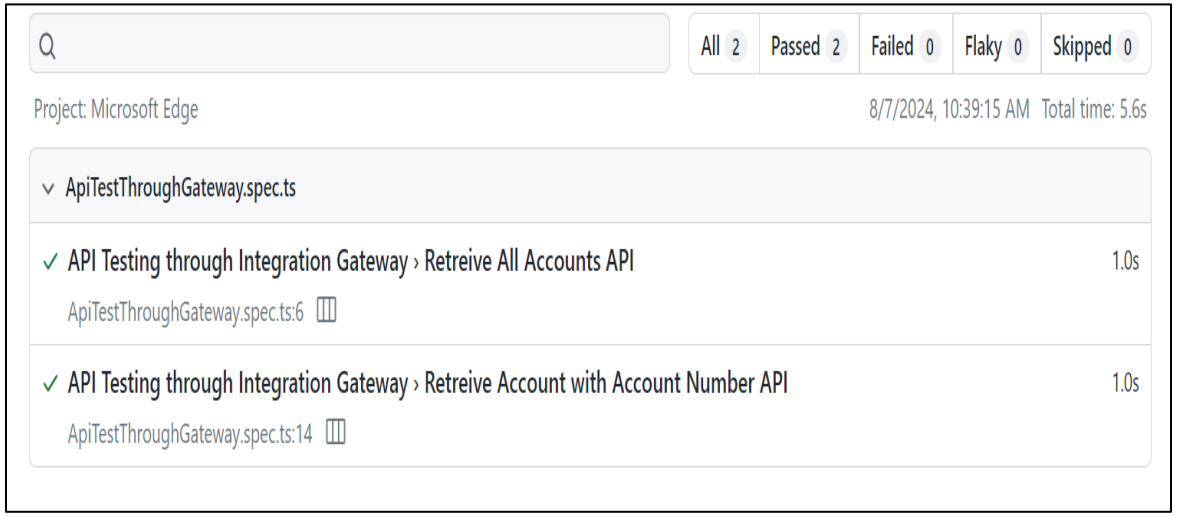
Cloud API testing through Integration Gateway functions with Playwrite:

- Create the account & Retrieve the account

(A custom cloud API has been developed in Integration Gateway. Playwright script was developed and executed to create an account, retrieve to view account summary through Integrated Gateway)

Execution Report:





Cloud API Testing through Integration Gateway with Playwright - 2

Purpose of this is to call 2 customized APIs (Create account API & Retrieve account API) through Integration Gateway and get response.

Create account API Request and Response JSONs:

createAccountReq.json

```
{
    "data": {
        "attributes": {
            "initialAccountHolder": {
                "accountingContact": true,
                "companyName": "chandu srinivas",
                "contactSubtype": "Company",
                "primaryAddress": {
                    "addressLine1": "Tma lane 3",
                    "city": "TestCity",
                    "country": "US",
                    "postalCode": "123456",
                    "state": {
                        "code": "AZ",
                        "name": "AZ"
                    }
                }
            },
            "initialPrimaryLocation": {
                "addressLine1": "Tma lane 3",
                "addressType": {
                    "code": "business",
                    "name": "Business"
                },
                "city": "Absarokee",
                "country": "US",
                "postalCode": "123456",
                "state": {
```

createAccountRes.json

```
{
  "data": {
    "attributes": {
      "accountHolder": {
        "displayName": "Fancy Feathers",
        "id": "pc:S6LULXJOQx67eHTar3vJO",
        "type": "AccountContact",
        "uri": "/account/v1/accounts/pc:SQgPtKBChdoOEWmxbN1xi/contacts/pc:S6LULXJOQx67eHTar3vJO"
      },
      "accountNumber": "0065381941",
      "accountStatus": {
        "code": "Pending",
        "name": "Pending"
      },
      "createdDate": "2024-09-02T10:42:13.860Z",
      "frozen": false,
      "id": "pc:SQgPtKBChdoOEWmxbN1xi",
      "numberOfContacts": "1",
      "preferredCoverageCurrency": {
        "code": "usd",
        "name": "USD"
      },
      "preferredSettlementCurrency": {
        "code": "usd",
        "name": "USD"
      },
      "primaryLanguage": {
        "code": "en_US",
```

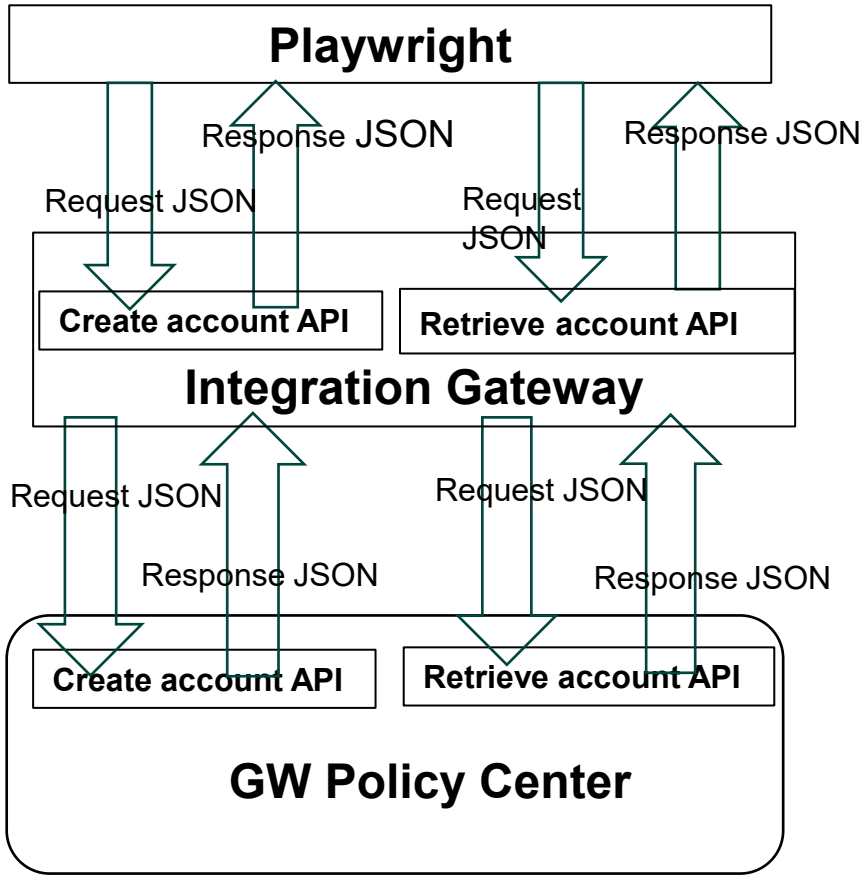
Retrieve account Request and Response JSONs:

Scenario : Retrieve Account from AcctNumber - IG Server
Method : GET
Endpoint URL :
http://localhost:8081/api/getAccountfromAcctNumber?accountNumber=C000456354

response_RetreiveAllAccounts.json

```
{
  "count": 25,
  "data": [
    {
      "attributes": {
        "accountHolder": {
          "displayName": "Wright Construction",
          "id": "test_pc:1",
          "type": "AccountContact",
          "uri": "/account/v1/accounts/pc:ds:1/contacts/test_pc:1"
        },
        "accountNumber": "C000212105",
        "accountStatus": {
          "code": "Active",
          "name": "Active"
        },
        "businessOperationsDescription": "business description",
        "createdDate": "2024-06-04T04:22:43.346Z",
        "frozen": false,
        "id": "pc:ds:1",
        "industryCode": {
          "code": "1522",
          "description": "General Contractors-Residential Building, Other Than Single-Family (except hotel and motel\n     construction)",
          "id": "SIC:1522"
        },
        "numberOfContacts": "8",
        "organizationType": {
```

Flow Diagram:



Test Data Parameterization – From External Source (Excel)

Playwright can read from Excel sheet and populates the data into the application.

Test Scenarios :Account Creation

Test Data:

	A	B	C
1	Fields	Values	Assertion Values
2		**PersonAccount**	
3	firstName	William	
4	lastName	Oliver	
5	country	United Kingdom	
6	town	London	
7	postalCode	E1W 2RG	
8	accountType	Person	
9	homePhone	+44 7975 111111	
10	workPhone	+44 7975 222222	
11	mobilePhone	+44 7975 333333	
12	primaryPhone	Home	
13	primaryEmail	williumOliver@gmail.com	
14	errorCodeMissingFilel		Address 1 : Missing required field
15	addressLine1	London	
16	addressType	Home	
17	organization	ACV Property Insurance	
18	producerCode	ProducerCode:16	
19	accountHolderName		William Oliver
20			

Tool comparison

Feature	Playwright	Selenium	GT Framework
Language Support	TypeScript, JavaScript, Python, .NET, Java	Java, C#, Ruby, Python, JavaScript	JavaScript, Test cafe, Node.js
Architecture	Chromium-based	Client-server	Built on Karate
Speed	Fast	Slow	Fast
Reporting	Built in Reports	Customizable	Customizable
Complexity	Moderate	High	Moderate
Debugging	Good	Challenging	Challenging
API Testing	Yes	Limited	Yes
Open Source	Yes	Yes	No

Pros and Cons

Advantages:

- Playwright supports all major browsers: Chromium, Firefox and Webkit.
- Supports different languages like: TypeScript, JavaScript, Python, .NET, and Java
- It is designed to be Fast (similar timeline of 18 seconds is observed when compared to GTF for one sample scenario), Reliable (in terms of system hang or cracking the application etc) and Maintainable.
- It has Auto-waits implementation which can eliminate usage of Explicit waits.
- It can simulate different devices and screen sizes.
- It Captures Video and Tracing for debugging.
- It can execute multiple test Cases parallelly on multiple workers.

Limitations:

- While Playwright is a powerful automation tool, it does have some limitations (From Beazley UWWB team context):
- Excel spreadsheets are not allowed for data parametrization. We need to use JSON format only.
- Limited knowledge and resources in the market.
- Review process is very stringent and requires approval to merge the code.
- Smaller Community: Compared to Selenium, Playwright has a smaller community, which might result in fewer resources and support.
- Unavailability of Automation SPOC to make any quick decisions and to provide support from Beazley's side.

Conclusion

- We were able to automate full policy life cycle with Playwright.
- Able to do API testing.
- Playwright POC has demonstrated significant potential for enhancing the test automation.

Appendix - 1

Playwright Overview

Playwright: A Modern Automation Framework

Playwright is a powerful, open-source automation library that allows you to control Chromium, Firefox, and WebKit with a single API. It's designed to be fast, reliable, and maintainable.

Key Features:

Cross-platform: Run tests on Windows, Linux, and macOS.

Cross-browser: Supports Chromium, Firefox, and WebKit.

Cross-language: Available in TypeScript, JavaScript, Python, .NET, and Java.

High performance: Executes tests rapidly and reliably.

Parallel Execution: Executes Test Cases Parallelly on multiple workers.

Modern web support: Handles modern web features like Web Workers, Service Workers, and more.

Auto-waiting: Eliminates the need for explicit waits.

Video and tracing: Captures test execution for debugging.

Mobile emulation: Simulates different devices and screen sizes.

Appendix - 2

Benefits of TypeScript

- **Type safety:** Prevents runtime errors and improves code quality.
- **Code readability and maintainability:** Enhances code structure and understanding.
- **Better tooling support:** Leveraging advanced IDE features.
- **Large community and ecosystem:** Access to libraries and resources.

By combining Playwright's automation capabilities with TypeScript's type safety and development features, you can create high-quality, efficient, and reliable test automation solutions.

Appendix - 3

Folder Structure

- **Playwright** and **TypeScript** together offer a powerful combination for building robust and maintainable automation test suites.
- **Folder Structure :**
- **Source folder** holds shared and Tests sub-folder which helps to organize **Shared utilities** and **Test suits** separately.
- The **'Data'** folder houses test data utilized in test cases. It accommodates test data in JSON or Excel file formats.
- **Fixtures** are essential in Playwright for establishing a consistent and controlled environment for your tests. They handle the setup and teardown of resources required for each test, ensuring isolation and reliability.
- **Pages** encapsulate web page-specific data, including element locators, page-related functions, and shared validation criteria.
- The **'Utils'** folder contains reusable functions shared across multiple pages.

